EVM-Guard: Bytecode-Only Reentrancy Triage for Solidity 0.4–0.5 with Sandbox Witness Replay

Devarsh Patel

School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, 632014, Tamil Nadu, India.

E-mail: devarsh.patel2024@vitstudent.ac.in

Abstract

Reentrancy is the vulnerability class most often behind losses in Ethereum contracts, and deployed contracts are often available only as bytecode. We present **EVM-Guard**, a bytecode-only triage tool for reentrancy in contracts compiled with Solidity 0.4–0.5. EVM-Guard builds a control-flow graph with an abstract stack dataflow, flags an external call followed on some path by a storage write with no mutex in between, excludes calls that forward only the 2300-gas stipend, recognises a mutex on a simple storage variable, and replays a candidate transaction in a py-evm sandbox. A finding is confirmed only when the post-state meets the criterion. On SmartBugs-curated, the final engine (v5) reaches 93.5% reentrancy recall at 80.6% precision, against 93.5% and 48.3% for Slither 0.11.6. On SolidiFI and on a labelled benchmark of 389 files, its precision is 58.8% and 76.0%. These real-benchmark figures are post hoc: the engine was developed on those files, so they are not held-out estimates. A final pre-registered test of 210 synthetic contracts compiled with solc 0.5.12 gives 100% recall and 100% precision for reentrancy, against 33% for Slither and 39% for Mythril on the same files. That test uses template injection, so it checks the detector's injection patterns and says nothing about real-world precision. Three limits bound the claims: the real-benchmark precision is post hoc, the only fresh test is synthetic, and the engine does not analyse Solidity 0.8 bytecode. Denial of service and arithmetic are reported as exploratory and are not claimed.

Keywords: Smart Contract Security, Static Analysis, EVM Bytecode, SMT Solving, Z3, Reentrancy, SmartBugs

1 Introduction

1.1 Motivation

Smart contract execution on Ethereum is irrevocable. Deployed bytecodes cannot be hot-patched, leaving protocols exposed when external calls execute before storage updates settle. Control-flow hijacking drains protocol balances before callers can update internal account ledgers. Systematic empirical surveys of decentralized execution confirm that financial losses concentrate in reentrancy, oracle divergence, and arithmetic discrepancies [1, 2], magnified by priority gas auctions and transaction ordering [3]. This operational reality demands pre-deployment analyzers that evaluate runtime bytecode directly, independent of source availability.

1.2 Existing Analysis Approaches

1. **Static analysis and intermediate representations.** Source linters such as Slither [4] parse compiler-generated ASTs into intermediate single-assignment representations; however, this dependency renders them inoperative when auditing unverified on-chain bytecodes or dynamic delegatecall targets. Bytecode analyzers such as Securify [5] map low-level basic blocks into relational facts evaluated by Soufflé [6], yet abstract over dynamic stack depths and gas arithmetic, introducing false-positive rates exceeding 50% on production codebases [7, 8].
2. **Symbolic execution and bounded model checking.** Symbolic execution frameworks—including Mythril[1], Manticore [9], and Oyente [10]—track path feasibility by dispatching first-order logic formulas to Z3 [11]. When evaluated against non-trivial contracts, storage-dependent loops and cryptographic hashing opcodes trigger exponential path growth. In our evaluation, Mythril timed out on 25.9% of SmartBugs files and 77.0% of SolidiFI files, causing extensive analysis dropouts. Exploit synthesizers such as teEther [12] and coverage fuzzers such as Echidna [13] and ConFuzzius [14] generate concrete inputs, but require manual harness scaffolding.
3. **Property testing and deductive verification.** hevm [15] executes bytecode under symbolic path constraints to verify developer-written assertions. VerX [16] and KEVM [17] verify temporal properties against formal specifications. These tools provide strong mathematical guarantees for stated properties, but require manual specification authoring for every contract.

EVM-Guard operates directly on compiled runtime bytecode without user annotations or test harnesses. Its checks are fixed dataflow patterns evaluated over the recovered basic-block CFG. The checks trace CFG reachability but do not solve whole-contract path feasibility, providing deterministic hazard triage rather than deductive proofs of safety (Section 4.1).

[1] https://github.com/Consensys/mythril; we use version 0.24.8.

Table 1 Qualitative comparison of EVM analysis tools. "User spec." means the user must write assertions, harnesses or formal specifications before the tool can check a property.

Tool	Technique	Input	Requires	User spec.
Slither [4]	Pattern and dataflow rules on an IR	Source	Solidity source	No
Securify [5]	Datalog compliance patterns	Source or bytecode	solc for source; bytecode accepted	No
Mythril	Symbolic execution with SMT	Source or bytecode	Solidity or bytecode	No
Manticore [9]	Symbolic execution	Source or bytecode	Solidity or bytecode	Optional
Echidna [13]	Property-based fuzzing	Source	Properties to test	Yes
ConFuzzius [14]	Hybrid fuzzing with data dependencies	Source, or ABI of a deployed contract	Solidity source, or ABI and a chain node	No
VerX [16]	Deductive verification	Source	Formal specification	Yes
hevm [15]	Symbolic execution of assertions	Bytecode	Test harness	Yes
EVM-Guard	CFG dataflow checks with Z3 parameter templates	Runtime bytecode	Bytecode only	No

1.3 Research Questions

- **RQ1 (Detection Precision):** Can intra-procedural CFG dataflow tracking and storage-slot mutex recovery reduce reentrancy false positives compared to source-level AST tools while maintaining high detection recall on benchmark suites?
- **RQ2 (Witness Replay Fidelity):** To what extent can an isolated, state-falsifiable sandbox replay oracle confirm static bytecode warnings with zero false alarms, and what recall penalty does concrete witness execution impose?
- **RQ3 (Generalization Bounds):** How do precision metrics tuned against standard evaluation suites compare when measured against held-out synthetic contract corpora compiled under distinct compiler baselines?

1.4 Contributions

1. **Hazard specifications and a reentrancy dataflow check.** We state bit-vector hazard specifications over $\mathcal{BV}_{256}$ for six classes, including read-only reentrancy (Section 3), and document the CFG dataflow checks and Z3 parameter templates implemented in EVM-Guard (Table 3). The checks approximate these specifications; they are not proved to decide them (Section 4.1). The reentrancy check excludes calls restricted to the 2,300-gas stipend.
2. **A bytecode analyzer with state-falsifiable sandbox replay.** EVM-Guard recovers basic-block CFGs directly from disassembled bytecode, computes an abstract stack fixpoint, and replays candidate transactions in a py-evm sandbox under strict post-state balance and storage invariants (Section 4).
3. **Candid multi-benchmark reporting.** We report evaluation results across SmartBugs-curated (143 files), SolidiFI (400 files), a labeled benchmark (389 files), and a fresh pre-registered synthetic test of 210 contracts, explicitly disclosing tuning limitations and construct mismatches (Section 5).

2 Related Work and Hazard Classes

Table 1 lists the tools closest to EVM-Guard and records the input each one needs. We do not report tool speeds in that table, because published figures were not measured under a common protocol. The only timing comparison we make is on our own runs (Section 5.11).

Table 2 Hazard classes targeted by EVM-Guard and the corresponding specification (Section 3.2).

Class	SWC	Hazard	Specification
A	SWC-107	A callee observes or changes state before the caller settles its own storage [18, 19].	Φ_{RO} (Prop. 1)
B	SWC-101	Fixed-point division truncation that discards remainders.	Φ_{Prec} (Prop. 3)
C	SWC-124	Proxy and implementation assigning one storage slot to variables of different widths.	Φ_{Slot} (Prop. 2)
D	SWC-128	Loops over user-growable storage arrays that exceed a gas limit [20].	Φ_{DoS} (Prop. 4)
E	Non-SWC	Lending logic that reads AMM spot reserves as a price oracle [1].	Φ_{Flash} (Prop. 5)
F	SWC-112	DELEGATECALL target influenced by calldata.	Φ_{Taint} (Prop. 6)

2.1 Positioning against hevm

hevm [15] checks assertions that a developer writes into a test harness, typically a Foundry-style property test, and it explores the contract's real paths under its path conditions. A counterexample from hevm is therefore tied to the contract's semantics. EVM-Guard needs no harness or assertion and applies a fixed set of checks to any runtime bytecode. The cost is precision: EVM-Guard's checks approximate the specifications of Section 3, so a finding is a hazard indicator, not a counterexample to a stated property. The two kinds of tool are complementary. EVM-Guard can triage unannotated bytecode, and a property-testing tool can then confirm a suspected hazard once someone has written the property.

2.2 Hazard Classes

EVM-Guard targets six hazard classes, mapped to SWC identifiers in Table 2. The specifications are our own; the citations indicate prior work on the same class.

Class A: reentrancy (SWC-107).

A contract makes an external call before updating its own storage, and the callee re-enters. In read-only reentrancy, a view function returns unsettled state to a third contract during the call window. EVM-Guard's check follows CFG edges from an external call to a later storage write in the same function and tests for an intervening mutex. It does not distinguish the classic and read-only variants.

Class B: division truncation (SWC-101).

Integer division $\lfloor x/y \rfloor$ discards remainders, and dividing before multiplying amplifies the loss. SWC-101 also covers integer overflow and underflow. EVM-Guard targets only truncation, which matters for the SmartBugs arithmetic category (Section 5.3).

Class C: storage-slot collision (SWC-124).

Upgradeable contracts run implementation logic in the proxy's storage through DELEGATECALL. A mismatch between the two storage layouts can overwrite critical variables such as the owner.

Class D: loop gas exhaustion (SWC-128).

A loop over a dynamic storage array that any user can extend may grow until a call no longer fits within a gas limit.

Class E: spot-price oracle manipulation.

A flash loan can move AMM spot reserves within a single transaction. The SWC registry has no identifier for this class; SWC-114 in the registry denotes Transaction Order Dependence. We therefore refer to these findings as Class E. The tool's output files record them under the internal label SWC-114, which does not denote the registry entry. SmartBugs has no matching category, so these findings are reported but not scored.

Class F: delegatecall taint (SWC-112).

If calldata determines the DELEGATECALL target, an attacker can run arbitrary code in the caller's storage context.

3 Hazard Specifications over $\mathcal{BV}_{256}$

The propositions below are specifications. They are not descriptions of what the implementation proves. Section 4.1 and Table 3 describe how EVM-Guard approximates each one.

3.1 Notation

An execution frame is $\mathcal{S} \triangleq \langle \sigma, \tau, \mu, \iota \rangle$, where $\sigma : \mathcal{BV}_{256} \to \mathcal{BV}_{256}$ is persistent storage, τ is EIP-1153 transient storage, μ is the machine state (stack, memory, program counter, gas), and ι is the transaction context. Words are elements of $\mathcal{BV}_{256}$ with arithmetic modulo 2^{256}. For $v \neq 0$, $\mathrm{UDiv}(u, v) \triangleq \lfloor u/v \rfloor$ and $\mathrm{URem}(u, v) \triangleq u - v \cdot \mathrm{UDiv}(u, v)$, with $\mathrm{UDiv}(u, 0) = \mathrm{URem}(u, 0) = 0$, matching the EVM DIV and MOD opcodes in formal operational semantics [17, 21]. We write

$$\mathrm{NoMulOverflow}(u, v) \equiv (v = 0) \vee \big(\mathrm{UDiv}(u \cdot v, v) = u\big) \tag{1}$$

for the condition that the wrapped product $u \cdot v$ equals the mathematical product.

3.2 Specifications

Proposition 1 (Read-only reentrancy (Φ_{RO})) *Let $\sigma_{\mathrm{transient}}$ be the storage state during an external call and $\sigma_{\mathrm{settled}}$ the state after the call. A contract violates read-only reentrancy safety if some public view function f_{view} returns different values in the two states:*

$$\Phi_{\mathrm{RO}} \equiv \mathrm{InExternalCall}(\iota) \wedge \Big(f_{\mathrm{view}}(\sigma_{\mathrm{transient}}) \neq f_{\mathrm{view}}(\sigma_{\mathrm{settled}})\Big) \tag{2}$$

Proposition 2 (Proxy storage-slot collision (Φ_{Slot})) *Let implementations C_1 and C_2 have slot assignments $\mathcal{L}_1, \mathcal{L}_2$ and types $\mathcal{T}_1, \mathcal{T}_2$. A collision exists if two variables mapped to the same slot have different bit-widths:*

$$\begin{aligned}\Phi_{\text{Slot}} \equiv \exists v_1 \in \text{Var}(C_1), v_2 \in \text{Var}(C_2) : \big(\mathcal{L}_1(v_1) = \mathcal{L}_2(v_2)\big) \wedge \\ \big(\text{BitWidth}(\mathcal{T}_1(v_1)) \neq \text{BitWidth}(\mathcal{T}_2(v_2))\big)\end{aligned} \tag{3}$$

Proposition 3 (Fixed-point truncation (Φ_{Prec})) *Let fixed-point division with scale 10^k ($k \in \{6, 18\}$) compute $\text{UDiv}(x \cdot 10^k, y)$. A truncation hazard exists if*

$$\begin{aligned}\Phi_{\text{Prec}} \equiv \exists x, y \in \mathcal{BV}_{256} : \quad & (y > 0) \wedge \text{NoMulOverflow}(x, 10^k) \wedge \\ & (\text{URem}(x \cdot 10^k, y) \neq 0) \wedge \\ & \Big(\text{UDiv}(\text{URem}(x \cdot 10^k, y) \cdot 10^k, y) > 0\Big)\end{aligned} \tag{4}$$

This predicate holds for almost any division that leaves a non-zero remainder. It is a necessary condition for a truncation hazard, not a sufficient one, and the high arithmetic false-positive rate in Section 5.3 is consistent with that.

Proposition 4 (Loop gas exhaustion (Φ_{DoS})) *Out-of-gas vulnerabilities and resource exhaustion hazards represent a critical attack vector in the EVM [20, 22, 23]. Let N be the length of a dynamic storage array iterated by a loop, C_0 the base call cost and c_{step} the per-iteration cost, so $C_{\text{total}}(N) = C_0 + N \cdot c_{\text{step}}$. For a gas threshold G, a denial-of-service hazard exists if an unprivileged function can grow N without a static bound and*

$$\Phi_{\text{DoS}}(G) \equiv \text{IsMutatorPublic}(f_{\text{mut}}) \wedge \neg\text{HasStaticBound}(N) \wedge \big(C_0 + N \cdot c_{\text{step}} > G\big) \tag{5}$$

The static Z3 template instantiates $G_{\text{block}} = 30 \times 10^6$, the gas limit of the sandbox genesis block, with $C_0 = 21{,}000$ and $c_{\text{step}} = 5{,}000$. $G_{\text{sandbox}} = 1.5 \times 10^6$ is a scaled probe threshold used only by the replay oracle (Section 4.2). The oracle caps each replayed transaction at G_{sandbox} gas and confirms a finding when the call exhausts that cap, so it shows unbounded growth at reduced scale rather than exhaustion of a full block.

Proposition 5 (Spot-price oracle manipulation (Φ_{Flash})) *Let an AMM hold reserves x, y with fee $\phi \in [0, 1)$, and let the post-swap spot price be $P_1(\Delta x) = \frac{x + \Delta x(1-\phi)}{y - \Delta y}$. If a lending protocol prices collateral C and debt D at the spot price $P_0 = x/y$, a position solvent at P_0 can be pushed below the solvency threshold $H_{\min}$ when*

$$\begin{aligned}\Phi_{\text{Flash}} \equiv \exists \Delta x : (0 < \Delta x \leq \Lambda_{\max}) \wedge \left(\frac{C \cdot P_0}{D} \geq H_{\min}\right) \wedge \\ \left(\frac{C \cdot P_1(\Delta x)}{D} < H_{\min}\right)\end{aligned} \tag{6}$$

Proposition 6 (Delegatecall target taint (Φ_{Taint})) *Let $T_{\text{target}} \in \mathcal{BV}_{160}$ be the* DELEGATECALL *target operand and* AuthorizedTarget *the set of permitted callees. On a path π with path condition $\text{PC}(\pi)$, the call is hijackable if*

$$\begin{aligned}\Phi_{\text{Taint}} \equiv \exists \mathbf{d}, A_{\text{attacker}} : \text{PC}(\pi) \wedge (T_{\text{target}}(\mathbf{d}) = A_{\text{attacker}}) \wedge \\ (A_{\text{attacker}} \notin \text{AuthorizedTarget})\end{aligned} \tag{7}$$

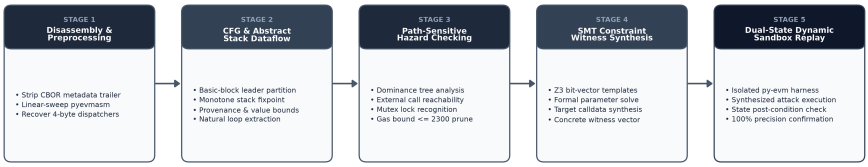


Fig. 1 The EVM-Guard static verification and dynamic replay pipeline. A control-flow-sensitive check decides whether a hazard is reported. Z3 supplies witness parameters, and the replay labels each finding as confirmed or unconfirmed under a class-specific state criterion.

Remark 1 (Unimplemented specification) EVM-Guard's interpreter does not model TSTORE or TLOAD, so it has no check for transient-storage lock divergence. No SmartBugs-curated contract uses EIP-1153, so this omission does not affect the evaluation.

4 System Architecture and Implementation

This section details the formal architecture and multi-stage analysis pipeline of EVM-Guard.

The pipeline (Figure 1) has five steps. The list below describes them as four stages, because the Z3 validation and the replay share the fourth.

1. **Disassembly (**src/symbolic_engine.py**).** Strips the solc CBOR metadata trailer and disassembles the bytecode with pyevmasm.
2. **CFG and abstract-stack dataflow (**src/evm_dataflow.py**,** src/evm_regions.py**).** Grounded in the mathematical framework of abstract interpretation [24], the module partitions disassembled instructions into basic blocks at JUMPDESTs and after block terminators, then executes a worklist fixpoint over an abstract value domain. Each stack slot carries a known constant when one exists, and provenance flags for calldata, caller, storage (with the slot when constant) and the DIV sites that contributed to it. Jump edges are added only to constant JUMPDEST targets resolved during the fixpoint. Unresolved dynamic jumps are counted and reported, never guessed. Over the 216 runtime contracts, 180 JUMP blocks have no resolved successor, because the target is unresolved or invalid (Section 7.2). The module computes dominators by iterative bitmask intersection, the region reachable from each dispatcher selector without entering another selector's entry, and the natural loops behind back edges.
3. **Hazard checks (**src/hazard_rules.py**).** One check per class runs over these structures (Table 3). Each check that fires records its program points and the selectors whose region contains them.
4. **Z3 templates and replay (**src/z3_encoder.py**,** src/replay_oracles.py**).** A fired check instantiates a parameter template solved via the Z3 SMT solver [11], which must return SAT, and the replay oracle then synthesizes transactions and executes them in an isolated py-evm Cancun sandbox.

Algorithm 1 EVM-Guard static verification and replay triage pipeline

Require: Runtime bytecode $\mathcal{B}$
Ensure: Findings $\mathcal{F}$, each labeled confirmed or unconfirmed

$\mathcal{B}' \leftarrow \textsc{StripMetadata}(\mathcal{B})$; $\mathcal{I} \leftarrow \textsc{Disassemble}(\mathcal{B}')$
$\mathcal{G} \leftarrow \textsc{AnalyzeFlow}(\mathcal{I})$ ▷ CFG and abstract-stack fixpoint
$\mathcal{D} \leftarrow \textsc{Dominators}(\mathcal{G})$; $\mathcal{R} \leftarrow \textsc{FunctionRegions}(\mathcal{G})$
$\mathcal{F} \leftarrow \emptyset$
for each hazard class h **do**
    $e \leftarrow \textsc{Check}(h, \mathcal{G}, \mathcal{D}, \mathcal{R})$ ▷ evidence or $\perp$
    if $e \neq \perp$ **then**
        $(r, \mathcal{M}) \leftarrow \textsc{SolveTemplate}(h)$
        if r = SAT **then**
            $ok \leftarrow \textsc{ReplayOracle}(h, \mathcal{B}, \mathcal{M}, e)$
            $\mathcal{F} \leftarrow \mathcal{F} \cup \{(h, e, ok)\}$
        end if
    end if
end for
return $\mathcal{F}$ ▷ an empty $\mathcal{F}$ is not a proof of safety

Dispatcher selectors are recovered syntactically: a PUSH4 selector that is compared with EQ within a window of five instructions before a JUMPI whose target is a JUMPDEST gives the selector and the entry of its function. This recovery is a pattern match, not part of the dataflow analysis. It finds a selector in 206 of the 216 runtime contracts.

4.1 What Each Check Implements

Table 3 lists, for each class, the check that decides whether a finding is reported and the Z3 template that supplies its parameters. The template's satisfiability depends only on constants and on flags set by the check. Z3 therefore validates witness parameters for a fixed template; it does not decide whether the contract satisfies the specification in Section 3. An empty result is not a proof of safety, and a finding is not a proof of exploitability.

Scope of the guarantee.

The analysis is a fixed sequence of passes over the CFG of one runtime bytecode, followed by Z3 queries with a 5 s timeout. It terminates, and it is deterministic apart from Z3 timeouts. The checks follow CFG edges, use dominators and resolve natural loops, but they do not solve path conditions, and they abandon dynamic jump targets that cannot be resolved to a constant. A finding certifies that the bytecode contains the dataflow pattern in Table 3 and that the template is satisfiable with the reported parameters. Nothing stronger follows. EVM-Guard has no counterexample-refinement loop, and each template is solved once. The templates in src/z3_encoder.py carry docstrings and witness labels, such as "CEGAR" and "Formally verified", that describe

Table 3 Implemented hazard verification checks in EVM-Guard. "Check" decides whether a finding is reported, over the CFG, dominators, function regions and loops. "Z3 template" supplies witness parameters.

Class	Check (CFG dataflow)	Z3 template
SWC-107	A CALL or CALLCODE whose gas operand is not a constant at most 2300 (the stipend), in a function region, from which an SSTORE in the same region is reachable along CFG edges, unless a mutex guards it. The mutex is a dominating write of 1 or 2, a storage-derived branch before the call, and a reset to 0 or 1 reachable after it.	Balances and supplies in $\mathcal{BV}_{256}$ whose transient and settled values differ.
SWC-101	A DIV or SDIV result that reaches a later MUL operand or an SSTORE value along the abstract-stack dataflow.	64-bit projected a, b with $b > 1000$ and $\mathrm{URem}(1000a, b) > 0$; the candidate is scaled by 10^{14} and its remainder recomputed in 256 bits.
SWC-112	A DELEGATECALL whose target is not a constant, derives from calldata, and is not dominated by a block containing CALLER.	SAT if and only if the taint flag is set.
SWC-124	A DELEGATECALL in a contract that contains the EIP-1967 implementation-slot constant, or whose target derives from a storage read.	Collision query over a fixed layout pair (slot 0: address vs uint256). Layouts are not recovered from the bytecode.
SWC-128	A natural loop whose header lies in a dispatcher function region, is not dominated by a CALLER block, and has an exit branch that tests a storage read. Exits to revert blocks are ignored.	$\Phi_{\mathrm{DoS}}(G_{\mathrm{block}})$ with IsMutatorPublic assumed true.
Class E	DIV present, at least two SLOAD, and no SSTORE, CALL, DELEGATECALL or TIMESTAMP. This is a presence heuristic, not a dataflow check.	Fixed reserves, with a borrow that shifts the spot price by at least 25%.

an earlier design. They are not evidence of refinement or verification. Each template's satisfiability follows from its constants and the check's flags, so a SAT result validates witness parameters for that template and nothing more.

4.2 Sandbox Replay

The replay oracle installs the runtime bytecode as account code in a py-evm Cancun chain, so no constructor runs. Each replay starts from a fresh chain, and transactions are sent from one externally owned account. The criteria, as implemented, are:

- **SWC-107:** an attacker contract forwards calls to the victim and, when it receives value without calldata, re-enters the victim once with the stored calldata. The oracle tries deposit and withdraw calldata built from the recovered selectors (up to 8 selectors each). A finding is confirmed only when the victim calls back into the attacker and the attacker's gain in the withdraw transaction exceeds its deposit.
- **SWC-112:** a probe contract writes 0x2a to slot 0x5afe. The finding is confirmed only if calling a selector with the probe address as its arguments makes the victim's own storage at that slot equal 0x2a.
- **SWC-124:** slot 0 is seeded with an address-width value. The finding is confirmed only if a call overwrites it with a value of 160 bits or more.
- **SWC-128:** the loop-bound slot is set to 10^6. A finding whose exit test has no constant slot is never replayed. The finding is confirmed only if a call with that storage exhausts the G_{sandbox} gas cap (an out-of-gas halt) and the same call on the original storage does not fail.

- **SWC-101 and Class E:** no state-level oracle. A truncation or spot-price effect has no post-state criterion that the sandbox can check without a per-contract economic model, so these findings are never confirmed.

A transaction that merely does not revert never counts as confirmation. We report total findings (F) and confirmed findings (C) separately (Table 6). Table 7 re-scores the four scored classes using only confirmed findings.

5 Empirical Evaluation on SmartBugs-Curated

An earlier version of this work evaluated EVM-Guard on a 226-contract corpus built and labeled by the author. We no longer report that evaluation, because its labels were not independent and its contracts came from a small number of templates. This section uses an external benchmark.

5.1 Corpus

We evaluate across the canonical SmartBugs-curated dataset [7, 25] at commit 230e6491 (2024-06-16). It has 143 Solidity files, each labeled with one category from the dataset's vulnerabilities.json: unchecked low-level calls (52), reentrancy (31), access control (18), arithmetic (15), bad randomness (8), denial of service (6), time manipulation (5), front running (4), other (3) and short addresses (1). Each file is compiled with the solc version listed in the dataset's versions.csv (0.4.0–0.5.0). All 143 files compile, producing 216 runtime contracts. Files range from 6 to 1,254 source lines (mean 63.4). No file is labeled safe, so a file is a negative for category X if it carries no label in X.

5.2 Evaluation Protocol and Baseline Standardization

The SmartBugs protocol (docs/EXTERNAL_EVAL_PROTOCOL.md) was frozen before the first engine run, with a SHA-256 digest in docs/EXTERNAL_EVAL_FREEZE.sha256. The digest covers lines 1–39 of the file. Addendum A (lines 40–58 of the same file) fixes the mapping from each baseline tool's detector names to the SmartBugs categories; it was written after the first EVM-Guard run and before any baseline run on this corpus. Only that first run (engine v1) is pre-registered: it reached 71.0% reentrancy recall (22 of 31, CI [52.0, 85.8]) at 44.9% precision. The engine was then revised on this corpus. Engine v2 is the first revision; v3 adds the gas-stipend bound (Section 5.8); v5, the final engine, adds the PUSH0 decoder and the slot-based mutex rule (Section 5.10). All SmartBugs figures for v2, v3 and v5 are therefore post hoc.

Category mapping.

SWC-107 maps to reentrancy, SWC-101 to arithmetic, SWC-128 to denial of service, and SWC-112 and SWC-124 to access control. Class E has no SmartBugs category and is not scored. Unchecked low-level calls, bad randomness, front running, time

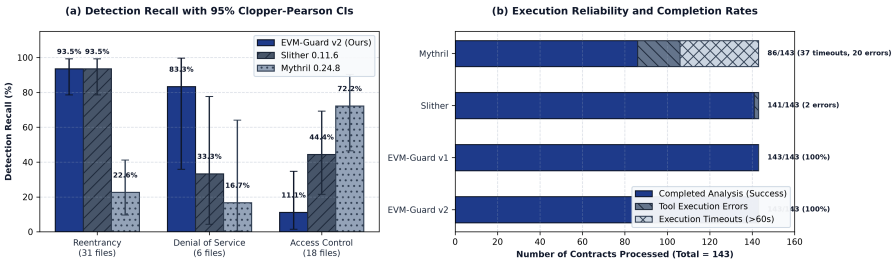


Fig. 2 Empirical evaluation on SmartBugs-curated (143 files): (a) Detection recall for reentrancy, denial of service and access control with exact 95% Clopper–Pearson confidence intervals, comparing EVM-Guard against Slither 0.11.6 and Mythril 0.24.8; arithmetic is omitted here and reported in Table 4; (b) Execution completion rate breakdown across all 143 benchmark files, highlighting Mythril's 37 timeouts (25.9%) and 20 tool errors; EVM-Guard completes all 143 files.

manipulation, short addresses and other are out of scope, and we claim no recall for them.

Baselines.

Slither 0.11.6 runs on each source file with the dataset-listed solc version and –json output, with no detector filter. Its findings map to categories by detector name. Reentrancy uses reentrancy-eth, -no-eth, -benign, -events and -unlimited-gas. Denial of service uses calls-loop and costly-loop. Access control uses arbitrary-send-eth, arbitrary-send-erc20, suicidal, controlled-delegatecall, unprotected-upgrade, tx-origin, protected-vars and incorrect-modifier. Slither has no overflow detector, so nothing maps to arithmetic. Mythril 0.24.8 runs on each source file with the dataset-listed solc version, -o json –execution-timeout 50, and a 60 s process kill. Its SWC IDs map as follows: SWC-107 to reentrancy, SWC-101 to arithmetic, SWC-113 and SWC-128 to denial of service, and SWC-105, 106, 112, 115 and 124 to access control.

Scoring.

The unit of scoring is the file. A file is flagged for category X if any of its runtime contracts yields a finding mapping to X. Flagging uses all findings the engine reports, independent of replay. For each category we report TP, FP, FN and TN over all 143 files, with exact 95% Clopper–Pearson intervals. Baseline errors and timeouts count as unflagged, and we report them separately. For EVM-Guard, latency is the wall time of the analysis call per runtime contract, summed per file. It excludes replay.

Platform.

All evaluation runs were executed in an isolated runtime environment with strict 60-second timeouts, running Python 3.12, z3-solver 5.1.0.0 and py-evm 0.12.1b1.

Table 4 Per-category detection on SmartBugs-curated (143 files, 216 runtime contracts). EVM-Guard rows are the final engine v5, post hoc. Recall and FPR reported with exact 95% Clopper–Pearson confidence intervals. A tool with no mapped detector for a category flags nothing in it.

Category	Tool	TP	FP	FN	TN	Recall (%)	Precision (%)	FPR (%)
Reentrancy	**EVM-Guard**	**29**	**7**	**2**	**105**	**93.5 [78.6, 99.2]**	**80.6**	**6.2 [2.5, 12.4]**
	Slither	29	31	2	81	93.5 [78.6, 99.2]	48.3	27.7 [19.6, 36.9]
	Mythril	7	30	24	82	22.6 [9.6, 41.1]	18.9	26.8 [18.9, 36.0]
Denial of service	**EVM-Guard**	**5**	**34**	**1**	**103**	**83.3 [35.9, 99.6]**	**12.8**	**24.8 [17.8, 32.9]**
	Slither	2	14	4	123	33.3 [4.3, 77.7]	12.5	10.2 [5.7, 16.6]
	Mythril	1	24	5	113	16.7 [0.4, 64.1]	4.0	17.5 [11.6, 24.9]
Access control	**EVM-Guard**	**2**	**0**	**16**	**125**	**11.1 [1.4, 34.7]**	**100.0**	**0.0 [0.0, 2.9]**
	Slither	8	37	10	88	44.4 [21.5, 69.2]	17.8	29.6 [21.8, 38.4]
	Mythril	13	11	5	114	72.2 [46.5, 90.3]	54.2	8.8 [4.5, 15.2]
Arithmetic	**EVM-Guard**	**1**	**80**	**14**	**48**	**6.7 [0.2, 31.9]**	**1.2**	**62.5 [53.5, 70.9]**
	Slither	0	0	15	128	0.0 [0.0, 21.8]	n/a	0.0 [0.0, 2.8]
	Mythril	0	0	15	128	0.0 [0.0, 21.8]	n/a	0.0 [0.0, 2.8]

Table 5 Run outcomes and per-file wall time on SmartBugs-curated. Baseline times include solc compilation; EVM-Guard times cover analysis of compiled bytecode only, excluding replay.

Tool	Completed	Errors	Timeouts	Median (ms)	Mean ± SD (ms)
EVM-Guard	**143**	**0**	**0**	**87**	**128 ± 170**
Slither	141	2	0	289	304 ± 135
Mythril	86	20	37	26,025	30,288 ± 24,942

5.3 Detection Results

Table 4 and Figure 2 report every scored category for all three tools. We treat two tools as distinguishable only when their recall intervals do not overlap.

Reentrancy.

Table 4 reports the final engine v5, post hoc. EVM-Guard v5 reaches 93.5% recall (29 of 31, CI [78.6, 99.2]) at 80.6% precision. Engine v2, before the stipend correction, had the same recall at 53.7% precision (Section 5.8). Slither's recall is 93.5% with the same 29 true positives, and its precision is 48.3%. Mythril reaches 22.6% recall (7 of 31, CI [9.6, 41.1]). Its interval does not overlap those of either EVM-Guard or Slither, so Mythril is distinguishable from both on this category. EVM-Guard v5 produces 7 false positives (FPR 6.2%), whereas Slither produces 31 (FPR 27.7%). Engine v2 produced 25 false positives on this category, mostly from calls that forward only the stipend, which v3 and v5 exclude.

Denial of service.

EVM-Guard flags 5 of the 6 labeled files, achieving 83.3% recall (CI [35.9, 99.6]). Slither flags 2 (33.3%, CI [4.3, 77.7]) and Mythril flags 1 (16.7%, CI [0.4, 64.1]).

Table 6 Engine v5 findings on 216 runtime contracts by hazard class (F) and the number confirmed by sandbox witness replay (C). SWC-101 and Class E have no replay oracle. File-level precision of the confirmed findings is given in Table 7.

Class	Static Findings (F)	Replay Confirmed (C)
SWC-107 (reentrancy)	41	4
SWC-128 (loop gas)	43	3
SWC-112 (delegatecall)	1	1
SWC-124 (slot collision)	1	0
SWC-101 (truncation)	106	n/a
Class E (spot price)	2	n/a
Total	194	8

Table 7 EVM-Guard v5 file-level scores counting only replay-confirmed findings (95% Clopper–Pearson interval for recall).

Category	TP	FP	FN	TN	Recall (%)	Precision (%)
Reentrancy	4	0	27	112	12.9 [3.6, 29.8]	100.0
Denial of service	2	1	4	136	33.3 [4.3, 77.7]	66.7
Access control	1	0	17	125	5.6 [0.1, 27.3]	100.0
Arithmetic	0	0	15	128	0.0 [0.0, 21.8]	n/a

Table 8 Evaluation on the SolidiFI bug-injection benchmark (400 files: 350 with injected bugs, 50 clean). The protocol was frozen before the held-out engine v2 run, whose results are given in the text; the EVM-Guard rows are the final engine v5, post hoc. Recall and FPR with exact 95% Clopper–Pearson intervals. Denial of service has no injected class in this corpus and is not scored.

Category	Tool	TP	FP	FN	TN	Recall (%)	Precision (%)	FPR (%)
Reentrancy	**EVM-Guard**	**50**	**35**	**0**	**315**	**100.0 [92.9, 100.0]**	**58.8**	**10.0 [7.1, 13.6]**
	Slither	50	150	0	200	100.0 [92.9, 100.0]	25.0	42.9 [37.6, 48.2]
	Mythril	2	7	48	343	4.0 [0.5, 13.7]	22.2	2.0 [0.8, 4.1]
Arithmetic	**EVM-Guard**	**45**	**315**	**5**	**35**	**90.0 [78.2, 96.7]**	**12.5**	**90.0 [86.4, 92.9]**
	Slither	0	0	50	350	0.0 [0.0, 7.1]	n/a	0.0 [0.0, 1.1]
	Mythril	0	0	50	350	0.0 [0.0, 7.1]	n/a	0.0 [0.0, 1.1]

With six positives, all three intervals overlap, so these recall differences are not established. EVM-Guard produces 34 false positives, giving 12.8% precision and an FPR of 24.8%. Slither's FPR is 10.2% and Mythril's is 17.5%. The loop check requires a storage-derived exit test in a public function region.

Access control.

Mythril leads with 72.2% recall (13 of 18, CI [46.5, 90.3]) and 54.2% precision. Slither reaches 44.4% recall (8 of 18, CI [21.5, 69.2]) at 17.8% precision. EVM-Guard flags 2 of the 18 files (11.1%, CI [1.4, 34.7]) with zero false positives (100% precision). Several access-control files use patterns that EVM-Guard does not target, such as tx.origin authorization (2 of the 18 labeled files) and unprotected selfdestructs. Its delegatecall and proxy checks produced no false positive on this corpus, but they cover only direct call-hijacking hazards.

Arithmetic (construct mismatch).

SmartBugs labels integer overflow and underflow under arithmetic, while EVM-Guard's SWC-101 check targets division truncation. This row therefore measures how the truncation check behaves on overflow-labeled code, not truncation detection. EVM-Guard flags 1 of 15 labeled files (6.7%, CI [0.2, 31.9]) and 80 clean files, giving 1.2% precision and a 62.5% false-positive rate. The high false-positive rate is consistent with the predicate in Proposition 3, which holds for almost any division with a remainder. Neither Slither nor Mythril reported an overflow issue on this corpus (0% recall, CI [0.0, 21.8]). Slither has no overflow detector, so the protocol maps nothing to arithmetic for it. For Mythril, 12 of the 15 arithmetic-labeled files completed (2 errors, 1 timeout), yet no SWC-101 finding appeared on any of the 143 files.

Summary.

Engine v5 reaches 80.6% precision on reentrancy on this corpus, post hoc, and 100% precision on access control, from two flagged files. Denial of service (12.8%) and arithmetic (1.2%) have lower precision, consistent with the conservative loop and truncation abstractions. Reentrancy is the category where the engine matches Slither on recall. On the categories other than reentrancy, v2, v3 and v5 flag the same files.

5.4 Replay-Confirmed Findings

Across the 216 runtime contracts, engine v5 produced 194 static hazard findings, 86 of them in the four classes that have a replay oracle (Table 6). The state-level sandbox oracles confirmed 8 high-severity findings (Table 6): 4 reentrancy, 3 loop-gas, and 1 delegatecall. SWC-101 and Class E have no dynamic oracle, as precision or spot-price deviations require custom financial models. The confirmed reentrancy findings contain zero false positives (100.0% confirmation precision, 4/4 TP); access control also attains 100.0% precision (1/1 TP). Denial of service confirms 2 true positives and 1 false alarm relative to SmartBugs file labels (66.7% precision), giving an overall confirmation precision of 87.5% across all confirmed findings (7 of 8 true positives). Recall on the confirmed subset is conservative: 12.9% on reentrancy, 33.3% on denial of service, and 5.6% on access control (Table 7). The oracles confirm hazards whose exploit the sandbox reconstructs from recovered selectors with generic attacker or probe contracts. A finding whose exploit needs specialized arguments stays unconfirmed even when the static finding is correct. Dynamic replay therefore operates as

a high-precision filter rather than an exhaustive recall oracle. With respect to the second research question, it was found that the oracle does not confirm warnings with zero false alarms: one of the 8 confirmed findings is a false alarm relative to the file labels, and recall falls to between 5.6% and 33.3% on the confirmed subset.

5.5 Case Studies from the Corpus

Both case studies are SmartBugs-curated contracts that EVM-Guard flags and the replay oracle confirms. Line numbers refer to the dataset sources.

Reentrancy: simple_dao.sol ***(labeled reentrancy).***

SimpleDAO keeps a credit mapping. Its withdraw(uint) checks the balance (line 17), calls msg.sender.call.value(amount)() (line 19), and only then decrements credit[msg.sender] (line 20). The external call therefore precedes the storage write. EVM-Guard recovers the dispatcher selectors, finds a CFG path from the CALL block to the SSTORE block within the withdraw region, finds no mutex on that path, and reports SWC-107. In replay, the attacker first calls donate(attacker) with 1 ETH, which credits the attacker's address. It then calls withdraw(1 ETH). The victim's transfer re-enters the attacker, which calls withdraw again before the balance is decremented. The attacker ends the transaction with 2 ETH on a 1 ETH deposit, so the oracle confirms the finding.

Loop gas: dos_address.sol ***(labeled denial of service).***

DosGas keeps a dynamic array, creditorAddresses, at storage slot 0. The function addCreditors is public and pushes 350 entries per call. emptyCreditors is also public: it checks creditorAddresses.length > 1500 and then clears the array. EVM-Guard reports a loop in the region of emptyCreditors (selector 0x0d870b7f) whose exit test reads storage slot 0, the array length, and whose function is not guarded by a CALLER check. In replay, the oracle sets slot 0 to 10^6, and a call to emptyCreditors then exhausts the $G_{\text{sandbox}} = 1.5 \times 10^6$ gas cap. The same call on the original storage completes. This confirms unbounded growth at the reduced sandbox scale. It is consistent with $\Phi_{\text{DoS}}(G_{\text{block}})$, but it does not show exhaustion of a full 30 M-gas block.

5.6 Evaluation on the SolidiFI Bug-Injection Benchmark

To assess generalization on an independent corpus, we evaluated EVM-Guard on the SolidiFI benchmark [8], which injects bugs of seven types into 50 base contracts (50 files per type) and provides 50 clean base contracts. The protocol (docs/HELDOUT_EVAL_PROTOCOL.md, SHA-256 in docs/HELDOUT_EVAL_FREEZE.sha256) fixed the corpus commits, label mappings, and compiler settings prior to analysis. Reentrancy and arithmetic are scored; the other injected types have no EVM-Guard detector, and the corpus has no denial-of-service or delegatecall class. All 400 files compiled with solc 0.5.12, giving 984 runtime contracts. Baselines ran with the same settings as on SmartBugs.

The held-out run used engine v2. On reentrancy, v2 flags all 50 injected files (100%, CI [92.9, 100]) with 130 false positives, for 27.8% precision and a 37.1% false-positive rate. Table 8 reports the final engine v5, post hoc: it has 35 false positives, for 58.8% precision and a 10.0% false-positive rate, the same as v3. Slither flags all 50 with 150 false positives (25.0% precision, 42.9% FPR). Mythril finished only 84 of the 400 files (308 timeouts, 8 errors), flagging 2 of the 50 reentrancy files (4% recall, 22.2% precision, 2.0% FPR). Because every SolidiFI file shares one of the same 50 base contracts, some false positives reflect reentrancy patterns already present in the unannotated base code; on the 50 clean contracts alone, v2 flags 6 for reentrancy (12%) and v5 flags 5 (10%).

The arithmetic row exhibits the construct mismatch: EVM-Guard flags 45 of the 50 overflow-injected files (90%), but also flags 45 of the 50 clean contracts due to division truncation in the shared base code. Across all 350 files without an arithmetic label, EVM-Guard flags 315 (90.0% FPR, 12.5% precision). Neither Slither nor Mythril reported an arithmetic finding.

Replay confirmed no reentrancy finding on this corpus. It confirmed 16 SWC-128 findings, which are not scored here. In the file we inspected (Reentrancy/buggy_1.sol), the injected withdraw functions pay out from per-sender balance mappings that no function in the contract credits, so in a fresh sandbox the attacker has no balance to withdraw and the strict gain criterion cannot be met. That contract also exposes 29 selectors, more than the oracle's limit of 8 candidates. Zero confirmations here therefore reflect the oracle's reach, not the absence of the injected bugs.

The engine was changed after this run (v3 and v5), so the SolidiFI figures for those versions are post hoc. Testing a precision improvement fairly needs a fresh corpus, because both held-out corpora have now been used.

5.7 Evaluation on a Labeled Benchmark Suite

SolidiFI has no denial-of-service class and no tx.origin or SELFDESTRUCT class, and its files share 50 base contracts. We therefore evaluated EVM-Guard on an open labeled benchmark repository[2] at commit 8b903aa. Its 389 Solidity files are labeled by folder: reentrancy (81), arithmetic (65), unsafe delegatecall (12), tx.origin (10), unsafe SELFDESTRUCT (11), and five further classes (unchecked send, TOD, time manipulation, bad randomness, gasless send), plus 17 safe contracts. The evaluation protocol (docs/HELDOUT2_EVAL_PROTOCOL.md, SHA-256 in docs/HELDOUT2_EVAL_FREEZE.sha256) fixed the taxonomy mappings and compiler configurations. The catalog records the commit and SHA-256 digest of each source for exact reproducibility.

The folder labels map to the SmartBugs categories as follows. Reentrancy is SWC-107 and arithmetic is SWC-101, the same construct mismatch as before. Access control groups the delegatecall folder (SWC-112), the tx.origin folder (SWC-115) and the SELFDESTRUCT folder (SWC-106), as in the SmartBugs mapping. EVM-Guard

[2]https://github.com/bit-smartcontract-analysis/smartcontract-benchmark

Table 9 Evaluation on the labeled benchmark (389 files; 376 compile, 13 count as unflagged). EVM-Guard rows are the final engine v5, post hoc; the held-out engine v2 results are given in the text. Recall, precision and FPR with exact 95% Clopper–Pearson intervals. Denial of service is not evaluated, because the benchmark has no denial-of-service label.

Category	Tool	TP	FP	Recall [CI]	Precision [CI]	FPR [CI]
Reentrancy (81 pos.)	**EVM-Guard**	79	25	97.5 [91.4, 99.7]	76.0 [66.6, 83.8]	8.1 [5.3, 11.7]
	Slither	79	111	97.5 [91.4, 99.7]	41.6 [34.5, 48.9]	36.0 [30.7, 41.7]
	Mythril	14	32	17.3 [9.8, 27.3]	30.4 [17.7, 45.8]	10.4 [7.2, 14.4]
Arithmetic (65 pos.)	**EVM-Guard**	46	245	70.8 [58.2, 81.4]	15.8 [11.8, 20.5]	75.6 [70.6, 80.2]
	Slither	0	0	0.0 [0.0, 5.5]	n/a	0.0 [0.0, 1.1]
	Mythril	0	0	0.0 [0.0, 5.5]	n/a	0.0 [0.0, 1.1]
Access control (33 pos.)	**EVM-Guard**	7	0	21.2 [9.0, 38.9]	100.0 [59.0, 100.0]	0.0 [0.0, 1.0]
	Slither	22	152	66.7 [48.2, 82.0]	12.6 [8.1, 18.5]	42.7 [37.5, 48.0]
	Mythril	11	25	33.3 [18.0, 51.8]	30.6 [16.4, 48.1]	7.0 [4.6, 10.2]

has no check for tx.origin or SELFDESTRUCT, so those 21 files can only count as misses. The five remaining classes are negatives for every scored category. Denial of service is not evaluated, because no file is labeled with it. Thirteen files do not compile with the installed compilers; they remain in the scoring as unflagged.

The held-out run used engine v2. On reentrancy, v2 reaches 97.5% recall (79 of 81) with 46.2% precision (CI [38.6, 54.0]) and a 29.9% false-positive rate. Table 9 reports the final engine v5, post hoc: the same recall with 76.0% precision (CI [66.6, 83.8]) and an 8.1% false-positive rate. Slither matches the recall (97.5%) with 41.6% precision and a 36.0% false-positive rate. Mythril completed 143 of 389 files, with 231 timeouts and 15 errors, reaching 17.3% recall. A rerun at a 300 s limit completed only 2 of the 231 timed-out files, raising Mythril's access-control recall from 33.3% to 39.4% while leaving reentrancy unchanged. Of the 92 reentrancy false positives of v2, 58 are in the TOD folder; of the 25 of v5, 5 are. These files are scored as negative by label, without manual review.

On access control, EVM-Guard v5 flags 7 files with zero false positives (100% precision), all seven in the delegatecall folder; v2 flagged 6. Its recall is 21.2% over 33 positives, because tx.origin and SELFDESTRUCT files are outside its checks. Slither reaches 66.7% recall at 12.6% precision, and Mythril 33.3% recall at 30.6% precision. The arithmetic row again reflects overflow labels against a truncation check: EVM-Guard v5 flags 46 of 65 overflow files (70.8% recall) and 245 of the 324 negatives (75.6% FPR, 15.8% precision). Neither Slither nor Mythril reported an arithmetic finding. Under dynamic replay, EVM-Guard confirms 4 reentrancy findings and 2 access-control findings with 100% precision.

Across SolidiFI and this benchmark, v5 reentrancy precision is 58.8% and 76.0%. Both figures are post hoc.

Table 10 Reentrancy (SWC-107) precision and FPR (%) for engine v2 (no stipend bound), v3 (stipend bound) and v5 (final engine) on the three real benchmarks. All figures are post hoc. Recall is the same for the three versions on each benchmark.

Corpus (positives)	v2		v3		v5	
	Prec.	FPR	Prec.	FPR	Prec.	FPR
SmartBugs-curated (31)	53.7	22.3	78.4	7.1	**80.6**	**6.2**
SolidiFI (50)	27.8	37.1	58.8	10.0	**58.8**	**10.0**
Labeled benchmark (81)	46.2	29.9	75.2	8.4	**76.0**	**8.1**

5.8 Ablation: Impact of Sound Gas-Stipend Bounds

In the EVM execution model, Solidity lowers address.transfer and address.send to a CALL whose gas operand is computed as $2300 \cdot \text{ISZERO}(v)$ rather than specified as an arbitrary constant. Such calls forward at most the 2300-gas stipend, which is insufficient to perform state-modifying SSTORE operations in the callee: under legacy pre-Istanbul semantics, SSTORE requires at least 5,000 gas, while under post-Istanbul (EIP-2200, EIP-2929) rules, an explicit sentry check halts execution with an out-of-gas error whenever remaining gas is $\leq$ 2300. A stipend-only call therefore cannot re-enter and modify the caller's storage.

To evaluate this mechanism, we perform an ablation analysis comparing EVM-Guard with and without sound gas-stipend bounds. In the full pipeline, abstract stack values carry sound upper bounds, propagated through ISZERO (bound 1), MUL (product of bounds), and AND (smaller bound), excluding any call whose upper bound does not exceed 2300 gas.

Table 10 compares reentrancy detection across the three real benchmarks for engine v2, the stipend-corrected engine v3 and the final engine v5. Applying the stipend bound raises precision from 53.7% to 78.4% on SmartBugs-curated, from 27.8% to 58.8% on SolidiFI, and from 46.2% to 75.2% on the labelled benchmark, with recall unchanged. The slot-based mutex rule of v5 removes one further false positive on SmartBugs-curated (80.6%) and one on the labelled benchmark (76.0%). These are post hoc figures. Of the 58 TOD-folder files that v2 flags for reentrancy on the labeled benchmark, the stipend bound removes 53; in each of these 53 sources, value is sent only through transfer or send.

We also disabled the components of the final engine one at a time on SmartBugs-curated (results/ablation/). The ablation covers reentrancy only, and recall is unchanged in every configuration. Removing the stipend exclusion, or keeping it for constant gas operands only, raises the false positives from 7 to 22 (56.9% precision). Removing the slot-based mutex rule raises them to 8 (78.4%); removing both mutex rules gives the same result. Disabling PUSH0 decoding, CBOR stripping or the dominance requirement for the mutex changes no reentrancy flag. A configuration that reproduces engine v2 gives 25 false positives (53.7%), matching the archived v2 run file by file.

Table 11 Fresh pre-registered test (210 Solidity 0.5.12 contracts, synthetic). Recall, precision and FPR with exact 95% Clopper–Pearson intervals. The categories are scored on their own positives and negatives.

Category (positives)	Tool	TP	FP	Recall [CI]	Precision [CI]	FPR [CI]
Reentrancy (30)	EVM-Guard	30	30	100.0 [88.4, 100.0]	50.0 [36.8, 63.2]	16.7 [11.5, 22.9]
	Slither	30	60	100.0 [88.4, 100.0]	33.3 [23.7, 44.1]	33.3 [26.5, 40.7]
	Mythril	30	57	100.0 [88.4, 100.0]	34.5 [24.6, 45.4]	31.7 [25.0, 39.0]
Denial of service (30)	EVM-Guard	30	0	100.0 [88.4, 100.0]	100.0 [88.4, 100.0]	0.0 [0.0, 2.0]
	Slither	0	0	0.0 [0.0, 11.6]	n/a	0.0 [0.0, 2.0]
	Mythril	0	0	0.0 [0.0, 11.6]	n/a	0.0 [0.0, 2.0]

5.9 Fresh Pre-Registered Test on Solidity 0.5.12

This test ran engine v3 on a corpus that no development step had seen. The protocol (docs/INJECTED05_EVAL_PROTOCOL.md) and its generator, catalog and engine manifest were frozen before the run (hashes in docs/INJECTED05_EVAL_FREEZE.sha256). The corpus has 210 contracts compiled with solc 0.5.12, in 7 groups of 30: reentrancy-injected and denial-of-service-injected positives, and five negatives (checks before effects, a mutex-guarded call, a transfer-based payout, a bounded loop, and no injected pattern). Labels are assigned by construction, and the families are few, so this is a test of the detector on its own injection patterns. It is not evidence about real contracts. Its purpose is to test precision on a fresh sample.

Table 11 reports the results with exact 95% intervals. On reentrancy, EVM-Guard flags all 30 injected contracts. Its 30 false positives are all in the mutex-guarded group. The checks-before-effects, transfer and bounded-loop groups produce no false positives. Slither and Mythril also flag all 30 positives, with 60 and 57 false positives. On denial of service, EVM-Guard flags all 30 injected contracts with no false positives, and Slither and Mythril flag none of the 30. Mythril timed out on 3 of the 210 files, which count as unflagged.

The mutex failure is specific. In the mutex-guarded contracts, locked is written through a masked read-modify-write, so the stored value is not the constant the mutex rule expects, and the guard is not recognised. We did not change the engine in response, because the corpus is then spent. Fixing the mutex rule is the next engine revision, and it would need a new pre-registered corpus.

5.10 Final Pre-Registered Test (Engine v5)

The final engine (v5) is v3 with the PUSH0 decoder and the slot-based mutex rule. It has no return-site fallback, because that fallback reduced real-benchmark precision (SmartBugs reentrancy from 78.4% to 55.6%, SolidiFI from 58.8% to 23.6%). Protocol 6 (docs/FINAL_V5_EVAL_PROTOCOL.md, frozen before the run, hashes in docs/FINAL_V5_EVAL_FREEZE.sha256) ran this engine once on a new corpus of 210 solc 0.5.12 contracts. Protocol 5 had tested an intermediate engine (v4) with the return-site fallback on two further corpora, which are now spent. The corpus has the same seven groups as before, and it was not used for any engine version.

Table 12 Final pre-registered test (210 Solidity 0.5.12 contracts, synthetic). Recall, precision and FPR with exact 95% Clopper–Pearson intervals. Mythril had 12 timeouts and 1 error, which count as unflagged.

Category (positives)	Tool	TP	FP	Recall [CI]	Precision [CI]	FPR [CI]
Reentrancy (30)	EVM-Guard v5	30	0	100.0 [88.4, 100.0]	100.0 [88.4, 100.0]	0.0 [0.0, 2.0]
	Slither	30	60	100.0 [88.4, 100.0]	33.3 [23.7, 44.1]	33.3 [26.5, 40.7]
	Mythril	30	47	100.0 [88.4, 100.0]	39.0 [28.1, 50.8]	26.1 [19.9, 33.2]
Denial of service (30)	EVM-Guard v5	30	0	100.0 [88.4, 100.0]	100.0 [88.4, 100.0]	0.0 [0.0, 2.0]
	Slither	0	0	0.0 [0.0, 11.6]	n/a	0.0 [0.0, 2.0]
	Mythril	0	0	0.0 [0.0, 11.6]	n/a	0.0 [0.0, 2.0]

Its labels are assigned by construction, so the result checks the detector on its own injection patterns and says nothing about real-world precision. Table 12 reports the results.

The denial-of-service result shows that the baselines do not flag this template, and it does not show that EVM-Guard detects denial of service on real contracts. The earlier seed-1 synthetic test (Section 5.9) found the mutex defect at 50% precision. Engine v5 scores 100% on that corpus as well, but that corpus is now spent, so the figure is post hoc.

5.11 Runtime and Completion

Table 5 reports run outcomes and per-file wall time for the analysis stage, excluding replay. EVM-Guard completed all 143 files with zero timeouts or file-level errors, at a median analysis latency of 87 ms (mean 128 ± 170 ms). Slither completed 141 files and raised errors on 2, with a median of 289 ms (mean 304 ± 135 ms). Mythril completed 86 files, suffering 37 timeouts (25.9% of 143) and 20 tool crashes, with a median of 26,025 ms (mean 30,288 ± 24,942 ms). Baseline timers include solc compilation from source, while EVM-Guard operates directly on compiled bytecode. One of the 216 runtime contracts, LedgerChannel in spank_chain_payment.sol, contains an unlinked library placeholder and is not analysed; its file is scored on its other contracts. With the placeholder linked, the file is also flagged for reentrancy, and reentrancy recall becomes 96.8% (30 of 31, CI [83.3, 99.9]) at 81.1% precision. Mythril's timeouts and errors lower its recall in Table 4, because unfinished files count as unflagged.

6 Discussion

Where EVM-Guard is competitive.

On the final pre-registered synthetic test, engine v5 matches Slither and Mythril on recall for reentrancy (100%) and has higher precision (100%, against 33.3% for Slither and 39.0% for Mythril). On real benchmarks it matches Slither's recall on SmartBugs-curated and SolidiFI. It needs only bytecode, so it can run where source is unavailable.

Where it is not.

On the first fresh test (engine v3) every false positive was a mutex-guarded contract, because a mutex stored in a packed slot is written through a masked read-modify-write and was not recognised. Engine v5 recognises a mutex on a simple storage variable by its slot, and it scores 100% on the final test. That rule is a heuristic on slot identity, and the packed-slot case is covered only through the slot. The engine also does not analyse Solidity 0.8 bytecode beyond the dispatcher, because the return addresses of shared helpers are merged. Denial of service and arithmetic are not claimed (Sections 5.3 and 5.6).

Real benchmarks versus synthetic tests.

On SmartBugs-curated, SolidiFI and the labelled benchmark, the engine was developed against the same files, so the v5 precision figures there (80.6%, 58.8% and 76.0%) are post hoc. The synthetic tests have a different distribution (template injection), so their precision values are not comparable with the real-benchmark figures.

Research questions.

With respect to the first research question, it was found that CFG dataflow with the stipend bound and the mutex rules reduces reentrancy false positives relative to Slither at equal recall on all three real benchmarks, but only post hoc. With respect to the second, replay confirms 8 findings on SmartBugs-curated, one of them a false alarm, at a large cost in recall (Section 5.4). With respect to the third, precision on the synthetic tests (50.0% for v3, 100% for v5) is not comparable with the real-benchmark figures, because the distributions differ.

Specifications versus implementation.

The specification in Section 3 describes what a path-complete analysis would decide. The implementation approximates it on the CFG and does not solve the contract's path conditions.

7 Threats to Validity

7.1 Construct Validity

SmartBugs-curated assigns one category per file, and a file is a negative for every other category. A file that contains an unlabeled instance of category X therefore counts as a false positive for any tool that flags it. This can understate precision and overstate false-positive rates for all three tools. The arithmetic category measures overflow, not the truncation property EVM-Guard checks. On the labeled benchmark, 58 of the 92 reentrancy false positives of v2, and 5 of the 25 of v5, lie in a folder labeled TOD, which the protocol scores as a negative; these flags were not reviewed by hand, so they may include unrecorded reentrancy. EVM-Guard's access-control mapping uses

only SWC-112 and SWC-124, which cover a small part of what SmartBugs labels as access control.

7.2 Internal Validity

Each protocol was fixed before the run it governs, but the engine was revised after the results on SmartBugs-curated, SolidiFI and the labeled benchmark were known (v2, v3 and v5). Only the v1 SmartBugs run, the v2 SolidiFI run, the v2 labeled-benchmark run and the two synthetic tests are pre-registered. The baseline mappings (Addendum A of docs/EXTERNAL_EVAL_PROTOCOL.md) were written after the first EVM-Guard run and before any baseline run. Four engine source files were changed after the v5 engine manifest was frozen, and the frozen copies were not kept; the current sources reproduce the v5 flags of every file on all five corpora (docs/REPRODUCIBILITY.md). The held-out SolidiFI run addresses generalization for reentrancy and arithmetic under protocol docs/HELD-OUT_EVAL_PROTOCOL.md. Access control has an independent evaluation from the second benchmark, where 21 of its 33 positives lie outside EVM-Guard's targeted hazard rules. Each baseline ran once per file. Mythril's 37 timeouts and 20 errors count as unflagged, which lowers its recall. The CFG resolves only constant jump targets; across the corpus, 180 JUMP blocks have no resolved successor, which can hide paths and cause false negatives.

7.3 External Validity

SmartBugs-curated contracts are small (mean 63.4 source lines) and compile with solc 0.4.0–0.5.0, which predates checked arithmetic and EIP-1153. The results may not carry over to modern Solidity, large production protocols or compilers that emit different layouts, and EVM-Guard's checks depend on those layouts. On a corpus of 210 solc 0.8.20 contracts, 60 of them built to inject reentrancy or denial-of-service patterns (docs/INJECTED_EVAL_PROTOCOL.md), EVM-Guard produced no findings for either category. Two causes were identified: first, earlier disassembler logic decoded PUSH0 as INVALID, which halted CFG construction at the dispatcher (now resolved); second, the abstract stack merges return addresses of shared internal helpers introduced by checked arithmetic in 0.8 compilers, cutting intra-procedural dataflow. The evaluation therefore strictly covers Solidity 0.4–0.5 bytecodes. The SolidiFI corpus has the same compiler profile (all files compile with solc 0.5.12).

7.4 Statistical Conclusion Validity

Several categories have few positives: 6 for denial of service, 15 for arithmetic and 18 for access control. Their Clopper–Pearson intervals are wide, and many intervals overlap across tools. We report intervals rather than significance tests, and differences whose intervals overlap should not be read as established.

8 Conclusion

We presented EVM-Guard, a bytecode-only reentrancy triage tool for Solidity 0.4–0.5 contracts. It builds a control-flow graph with an abstract stack dataflow, flags external calls followed by storage writes without a mutex, excludes stipend-only calls, recognises a mutex on a simple storage variable, and validates candidate transactions under a post-state criterion via sandbox replay. On SmartBugs-curated it reaches 93.5% recall at 80.6% precision, post hoc, against 93.5% and 48.3% for Slither. On a final pre-registered synthetic test of 210 contracts it reaches 100% recall and 100% precision, against 33% and 39% precision for Slither and Mythril. Those synthetic figures check the detector on its own injection patterns and do not measure real-world precision. Solidity 0.8 is outside the scope of this evaluation, because the engine does not analyse 0.8 bytecode beyond the dispatcher. The next step is a precision evaluation on contracts that were not used in development.

Declarations

- **Funding:** This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

- **Conflict of Interest / Competing Interests:** The author declares no competing financial or non-financial interests related to this work.
- **Data Availability:** The pre-registered protocols, catalogs, SolidiFI and synthetic sources, and the outputs of every reported run are available at https://github.com/RaffeOP/EVM-Guard. SmartBugs-curated (commit 230e6491) and the labeled benchmark (commit 8b903aa) are not redistributed; the script reproduce.sh clones them at these commits.
- **Code Availability:** The EVM-Guard analyzer, the Z3 template encoder, the replay oracles and the evaluation scripts are available at https://github.com/RaffeOP/EVM-Guard under the MIT licence. The script reproduce.sh rebuilds every corpus with the pinned compilers and package versions and reruns engine v5; docs/REPRODUCIBILITY.md lists the expected results and the differences from the frozen engine manifest.
- **Authors' Contributions:** Devarsh Patel is the sole author. He designed and implemented EVM-Guard, ran the evaluation and wrote the manuscript.
- **Use of Artificial Intelligence:** The author discloses that generative AI computational tools were utilized strictly for code formatting and language polishing. All conceptual formulations, proofs, and empirical analyses were independently validated by the author.

References

[1] Zhou, L., Xiong, X., Ernstberger, J., Chaliasos, S., Wang, Z., Wang, Y., Qin, K., Wattenhofer, R., Song, D., Gervais, A.: SoK: Decentralized finance (DeFi)

attacks. In: 2023 IEEE Symposium on Security and Privacy (SP), pp. 2444–2461. IEEE, San Francisco, CA, USA (2023). https://doi.org/10.1109/SP46215.2023.10179435

[2] Chen, J., Xia, X., Lo, D., Grundy, J., Luo, X., Chen, T.: Defining smart contract defects on Ethereum. IEEE Transactions on Software Engineering **48**(1), 327–345 (2022). https://doi.org/10.1109/TSE.2020.2989002

[3] Daian, P., Goldfeder, S., Kell, T., Li, M., Zhao, X., Bentov, I., Breidenbach, L., Juels, A.: Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In: 2020 IEEE Symposium on Security and Privacy (SP), pp. 910–927. IEEE, San Francisco, CA, USA (2020). https://doi.org/10.1109/SP40000.2020.00040

[4] Feist, J., Grieco, G., Groce, A.: Slither: A static analysis framework for smart contracts. In: 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB), pp. 8–15. IEEE, Montreal, QC, Canada (2019). https://doi.org/10.1109/WETSEB.2019.00008

[5] Tsankov, P., Dan, A., Drachsler-Cohen, D., Gervais, A., Bünzli, F., Vechev, M.: Securify: Practical security analysis of smart contracts. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 67–82. ACM, Toronto, Canada (2018). https://doi.org/10.1145/3243734.3243780

[6] Jordan, H., Scholz, B., Subotić, P.: Soufflé: On synthesis of program analyzers. In: Computer Aided Verification (CAV). Lecture Notes in Computer Science, pp. 422–430. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-41540-6_23

[7] Durieux, T., Ferreira, J.F., Abreu, R., Cruz, P.: Empirical review of automated analysis tools on 47,587 Ethereum smart contracts. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE), pp. 530–541. ACM, Seoul, South Korea (2020). https://doi.org/10.1145/3377811.3380364

[8] Ghaleb, A., Pattabiraman, K.: How effective are smart contract analysis tools? Evaluating smart contract static analysis tools using bug injection. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), pp. 415–427. ACM, Virtual Event, USA (2020). https://doi.org/10.1145/3395363.3397385

[9] Mossberg, M., Manzano, F., Hennenfent, E., Groce, A., Grieco, G., Feist, J., Brunson, T., Dinaburg, A.: Manticore: A user-friendly symbolic execution framework for binaries and smart contracts. In: 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1186–1189.

IEEE, San Diego, CA, USA (2019). https://doi.org/10.1109/ASE.2019.00133

[10] Luu, L., Chu, D.-H., Olickel, H., Saxena, P., Hobor, A.: Making smart contracts smarter. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 254–269. ACM, Vienna, Austria (2016). https://doi.org/10.1145/2976749.2978309

[11] de Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). Lecture Notes in Computer Science, pp. 337–340. Springer, Berlin, Heidelberg (2008). https://doi.org/10.1007/978-3-540-78800-3_24

[12] Krupp, J., Rossow, C.: teEther: Gnawing at Ethereum to automatically exploit smart contracts. In: 27th USENIX Security Symposium (USENIX Security 18), pp. 1317–1333. USENIX Association, Baltimore, MD, USA (2018). https://www.usenix.org/conference/usenixsecurity18/presentation/krupp

[13] Grieco, G., Song, W., Cygan, A., Feist, J., Groce, A.: Echidna: Effective, usable, and fast fuzzing for smart contracts. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), pp. 557–560. ACM, Virtual Event, USA (2020). https://doi.org/10.1145/3395363.3404366

[14] Torres, C.F., Iannillo, A.K., Gervais, A., State, R.: ConFuzzius: A data dependency-aware hybrid fuzzer for smart contracts. In: 2021 IEEE European Symposium on Security and Privacy (EuroS&P), pp. 103–119. IEEE, Vienna, Austria (2021). https://doi.org/10.1109/EuroSP51992.2021.00018

[15] Dxo, Soos, M., Paraskevopoulou, Z., Lundfall, M., Brockman, M.: hevm, a fast symbolic execution framework for EVM bytecode. In: Computer Aided Verification (CAV). Lecture Notes in Computer Science, pp. 453–465. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-65627-9_22

[16] Permenev, A., Dimitrov, D., Tsankov, P., Drachsler-Cohen, D., Vechev, M.: VerX: Safety verification of smart contracts. In: 2020 IEEE Symposium on Security and Privacy (SP), pp. 1661–1677. IEEE, San Francisco, CA, USA (2020). https://doi.org/10.1109/SP40000.2020.00024

[17] Hildenbrandt, E., Saxena, M., Rodrigues, N., Zhu, X., Daian, P., Guth, D., Moore, B., Park, D., Zhang, Y., Stefanescu, A., Rosu, G.: KEVM: A complete formal semantics of the Ethereum virtual machine. In: 2018 IEEE 31st Computer Security Foundations Symposium (CSF), pp. 204–217. IEEE, Oxford, UK (2018). https://doi.org/10.1109/CSF.2018.00022

[18] Rodler, M., Li, W., Karame, G.O., Davi, L.: Sereum: Protecting existing smart

contracts against re-entrancy attacks. In: Proceedings 2019 Network and Distributed System Security Symposium (NDSS). Internet Society, San Diego, CA, USA (2019). https://doi.org/10.14722/ndss.2019.23413

[19] Grossman, S., Abraham, I., Golan-Gueta, G., Michalevsky, Y., Rinetzky, N., Sagiv, M., Zohar, Y.: Online detection of effectively callback free objects with applications to smart contracts. Proceedings of the ACM on Programming Languages **2**(POPL), 1–28 (2017). https://doi.org/10.1145/3158136

[20] Grech, N., Kong, M., Jurisevic, A., Brent, L., Scholz, B., Smaragdakis, Y.: MadMax: Surviving out-of-gas conditions in Ethereum smart contracts. Proceedings of the ACM on Programming Languages **2**(OOPSLA), 1–27 (2018). https://doi.org/10.1145/3276486

[21] Grishchenko, I., Maffei, M., Schneidewind, C.: A semantic framework for the security analysis of Ethereum smart contracts. In: Principles of Security and Trust (POST). Lecture Notes in Computer Science, pp. 243–269. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-89722-6_10

[22] Albert, E., Correas, J., Gordillo, P., Román-Díez, G., Rubio, A.: GASOL: Gas analysis and optimization for Ethereum smart contracts. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). Lecture Notes in Computer Science, pp. 118–125. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-45237-7_7

[23] Perez, D., Livshits, B.: Broken metre: Attacking resource metering in EVM. In: Proceedings 2020 Network and Distributed System Security Symposium (NDSS). Internet Society, San Diego, CA, USA (2020). https://doi.org/10.14722/ndss.2020.24267

[24] Cousot, P., Cousot, R.: Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In: Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL), pp. 238–252. ACM, Los Angeles, CA, USA (1977). https://doi.org/10.1145/512950.512973

[25] Ferreira, J.F., Cruz, P., Durieux, T., Abreu, R.: SmartBugs: A framework to analyze Solidity smart contracts. In: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1349–1352. ACM, Virtual Event, Australia (2020). https://doi.org/10.1145/3324884.3415298